

1on1 --- Technology Stack & Engineering Plan

1. Current Technology Stack

Frontend

- React
- TypeScript
- HTML
- CSS
- anime.js
- D3.js

Backend

- Node.js
- Express
- TypeScript

Database

- MongoDB Atlas
- Mongoose

Realtime

- Socket.IO
- WebRTC

AI

- Gemini API
- Agent/tool calling
- Guardrails
- Prompt engineering
- AI moderation
- AI meeting notes

Version Control

- Git
- GitHub

hosting target

- Google Cloud Platform

Later infrastructure

- Redis
- RAG
- Conversation memory
- Background job processing
- Caching
- Presence scaling

Docker is intentionally excluded from the current plan.

2. Programming Languages

Required:

- TypeScript
- JavaScript concepts
- HTML
- CSS
- MongoDB/Mongoose query concepts

The application does not require multiple backend programming languages.

TypeScript should be preferred for both frontend and backend to reduce context switching and improve type safety.

3. Authentication

V1 authentication should support:

- Email
- Password
- JWT
- Access token
- Refresh token
- HTTP-only cookies
- Forgot password
- Password reset

There is **no phone-number authentication**.

Authentication identity is based on email + password.

Google authentication/OAuth can be included as an additional login option if required, but email/password remains the core credential flow.

Clerk is a future option and should not be introduced into the current implementation.

4. JWT Model

Recommended flow:

Login

User submits email/password.

Backend:

1. Find user.
2. Compare password hash.
3. Generate access token.
4. Generate refresh token.
5. Set secure HTTP-only refresh cookie.
6. Return authenticated application state.

Access token should be short-lived.

Refresh token should be longer-lived.

Refresh token rotation and server-side invalidation can be implemented when the auth layer is hardened.

5. Password Security

Passwords should never be stored directly.

we use hashing strong algo which is

- bcrypt

The database stores only the password hash.

6. Authentication Provider Roadmap

Current:

Custom JWT authentication

Later:

Clerk

Clerk should only be introduced when there is a clear benefit in reducing authentication infrastructure complexity.

Do not mix Clerk and custom JWT unnecessarily in V1.

7. REST API

Express provides REST APIs for:

- Authentication
- Users
- Profiles
- Follow system
- Posts
- Comments
- Search
- Sessions
- Availability
- Reviews
- Notifications
- Certifications
- Achievements
- AI tools

The frontend should not directly access MongoDB.

8. Socket.IO Responsibilities

Socket.IO should handle realtime events.

Messaging

- 1:1 messages
- Group/session chat
- Typing indicators
- Delivery status
- Seen status

Presence

- Online
- Offline
- Last seen

Notifications

Realtime notification delivery.

Meeting collaboration

- Whiteboard operations
- Reactions
- Raise hand
- Participant state
- Session events

WebRTC signaling

Socket.IO exchanges:

- SDP offers
- SDP answers
- ICE candidates

Socket.IO does NOT transmit the actual video/audio stream.

WebRTC handles media transmission.

9. Messaging Model

A message lifecycle:

Sent → **Delivered** → **Seen**

The actual message should be persisted in MongoDB.

Temporary realtime states such as typing indicators should not be persisted as normal messages.

A conversation can be:

- 1:1
- Group/session-based

Recommended collections:

- Conversation
 - Message
-

10. Online/Offline Presence

Socket.IO connection state provides initial presence.

For a single backend instance:

MongoDB may not be required for transient online status.

Later, when multiple backend instances exist, Redis should manage shared presence.

Future architecture:

Client → Socket.IO instance → Redis adapter → other Socket.IO instances

11. WebRTC

WebRTC handles:

- Audio
- Video
- Screen sharing
- Group session media

Socket.IO acts as the signaling mechanism.

Architecture:

Peer A → Socket.IO signaling → Peer B

After signaling:

Peer A ↔ WebRTC ↔ Peer B

For group calls, a pure peer-to-peer mesh can become inefficient as participant count increases.

The future scalable architecture can introduce an SFU such as mediasoup/LiveKit/Janus if needed.

Do not introduce an SFU into the prototype unless group-session scale requires it.

12. Meeting Features

V1 meeting:

- Video
- Audio
- Camera toggle
- Mic toggle
- Screen sharing
- Participant list
- Fullscreen
- Session chat
- Reactions
- Raise hand
- Whiteboard
- Shared notes
- Code snippets
- Polls
- Q&A
- Recording
- AI transcript
- AI summary
- Action items

No file sharing.

13. Whiteboard Technology

V1 can use a browser canvas-based whiteboard.

Realtime synchronization can use Socket.IO.

A whiteboard operation can be represented as an event:

- create object
- move object
- resize object

- delete object
- draw stroke
- edit text

For high-concurrency collaboration, a CRDT architecture can be introduced later.

15. Recording

Recording architecture should be treated separately from WebRTC signaling.

Browser-side recording can use MediaRecorder for an initial implementation.

Future cloud architecture can include:

- Object storage
- Transcoding
- Recording metadata
- Access permissions
- Retention policies

GCP storage is a future deployment consideration.

16. MongoDB Data Model

Core collections:

- User
- Post
- Comment
- Follow
- Conversation
- Message
- Notification
- Session
- SessionParticipant
- Availability
- Review
- Certification
- CertificateVerification
- Submission
- Badge

- Report
- MeetingTranscript
- MeetingNotes

Additional collections can be introduced only when the data model actually requires them.

17. User Collection

The User document should contain stable profile data:

- Name
- Email
- Password hash
- Avatar
- Cover
- Bio
- Skills
- Experience
- Education
- Profile metadata
- Provider/mentor status
- Verification status
- CreatedAt
- UpdatedAt

Do not store unlimited followers, following IDs, messages or posts directly inside User.

Those should be separate collections.

18. Follow Collection

A follow relationship:

- followerId
- followingId
- createdAt

Unique compound index:

`followerId + followingId`

Actions:

- Follow
- Unfollow
- Remove follower

Block relationships should be represented separately or through a dedicated relationship state.

19. Session Collection

Important fields:

- providerId
- title
- description
- type
- duration
- price
- currency
- capacity
- availability configuration
- status
- cancellation policy
- createdAt

Booking/session instances should be separate from reusable session offerings if the product supports recurring availability.

20. Session State Machine

Valid states:

REQUESTED

→ ACCEPTED

→ IN_PROGRESS

→ COMPLETED

Alternative paths:

REQUESTED → REJECTED

ACCEPTED → RESCHEDULED

ACCEPTED → CANCELLED

ACCEPTED → NO_SHOW

The backend must enforce legal state transitions.

21. Mock Payment Model

No real payment provider.

The system should simulate:

- Payment creation
- Payment success
- Payment failure
- Refund
- Cancellation fee

This lets the UI demonstrate paid-session behavior without handling real money.

22. Cancellation Engine

The backend calculates:

`timeUntilSession`

Then applies the configured cancellation rule.

Output:

- Original amount
- Fee
- Refund
- Rule

The frontend should display the backend-calculated result rather than calculate it independently.

23. AI Architecture

Gemini should sit behind a server-side AI service.

Frontend:

React → Express AI endpoint → Gemini

Never expose secret Gemini API keys in browser code.

24. Agent Tool Architecture

Tools are backend functions.

Examples:

- searchUsers()
- searchPosts()
- searchSessions()
- getAvailability()
- requestSession()
- cancelSession()
- rescheduleSession()
- followUser()
- unfollowUser()
- sendMessage()
- getNotifications()

Gemini decides which tool is relevant.

Express validates authorization.

The tool performs the action.

MongoDB stores the result.

25. AI Guardrails

The agent requires:

- System prompt
- Tool schemas
- Input validation
- Output validation
- Authorization
- Confirmation gates
- Rate limits
- Audit logging

- Prompt-injection defenses

The agent must not bypass backend authorization.

26. AI Moderation

AI moderation can run before publication or after content creation depending on latency requirements.

Content candidates:

- Posts
- Comments
- Messages where policy requires it
- Session descriptions
- Profile content

Moderation output should be structured:

- category
- risk level
- action
- confidence
- reason

Do not allow arbitrary model text to directly control database operations.

27. AI Meeting Intelligence

Meeting pipeline:

Meeting → transcript → Gemini → structured JSON → MongoDB

Store:

- transcript metadata
- summary
- key points
- decisions
- action items
- topics

Avoid storing raw audio indefinitely unless there is a clear retention policy.

28. AI Memory

Conversation memory is planned, not V1.

Later memory can include:

- User preferences
- Previous AI conversations
- Relevant session history
- Repeated tasks
- User profile context

Sensitive data should not automatically become permanent memory.

29. RAG

RAG is planned for later.

Potential sources:

- Platform documentation
- Feature documentation
- Cancellation policies
- User-approved profile information
- Session information
- AI assistant knowledge

RAG should not be introduced just to make the project sound more advanced.

30. Redis

Redis is planned for later.

Potential uses:

- Cache
- Socket.IO adapter
- Presence
- Rate limiting
- Session reminders
- Background jobs
- AI task queues

- Temporary state

MongoDB remains the primary database.

31. Background Jobs

Later, Redis + BullMQ can process:

- Session reminders
- AI meeting processing
- Email notifications
- Recording processing
- Moderation queues
- Certificate processing

Do not introduce a queue until synchronous processing becomes a problem.

32. Blockchain Certification

Certification verification is part of the current product plan.

Flow:

Certificate data → canonical representation → SHA-256 hash → certificate verification ID → verification record → blockchain hash anchoring

Sensitive certificate data should remain off-chain.

Only the cryptographic fingerprint or appropriate proof should be anchored.

33. GCP Hosting Roadmap

Initial prototype can run on Render.

Later target:

- GCP application hosting
- GCP storage for recordings/assets
- Gemini API / Google Cloud integration
- MongoDB Atlas remains an external managed database unless a future architecture decision changes it
- Redis as managed/hosted infrastructure later

Do not move to GCP prematurely just because it is the eventual hosting target.

34. No Docker in Current Plan

Docker is intentionally excluded.

The current objective is rapid development and deployment using the existing runtime.

Docker can be reconsidered later if:

- deployment complexity increases
 - multiple services need reproducible environments
 - workers are introduced
 - infrastructure becomes more complex
-

35. Testing Strategy

V1 should include practical tests around the most dangerous logic:

- Auth
- JWT validation
- Follow/unfollow
- Session state transitions
- Availability conflicts
- Cancellation calculations
- Mock refunds
- AI tool authorization

Realtime/WebRTC should be tested manually and with targeted integration tests.

36. Development Priority

Priority 1:

- Authentication
- Profiles
- Follow
- Posts/feed
- Search

Priority 2:

- Session creation
- Availability
- Session request
- Scheduling
- Mock payments
- Cancellation

Priority 3:

- WebRTC
- Group sessions
- Socket.IO chat
- Whiteboard
- Screen sharing

Priority 4:

- AI meeting notes
- AI moderation
- AI assistant
- Agent tools

Priority 5:

- Certifications + SHA-256 + blockchain anchoring
- Advanced achievements

Priority 6:

- Redis
- RAG
- Memory
- Background jobs
- GCP migration

37. Engineering Rule

Do not implement infrastructure before the product needs it.

The stack should grow in this order:

MongoDB → Socket.IO/WebRTC → Gemini → Redis → RAG/memory → scalable media infrastructure

This prevents the prototype from becoming an infrastructure project instead of a product.